Full project explanation

Architecture, rules, design decisions and implementation of FDSK-169 and FDSK-170

1. Background and the assignment

The take-home ("Business Rules Engine", Tejas) gave me an existing CRM codebase (AviKKi/field-desk) and two tickets. The point was not only to ship the features but to show how I decide how much structure to add: invest in a rules framework, or find a middle ground. Deliverables included a private repo with un-squashed history, architecture documents showing how the design evolved, a decision tree for when to invest in a framework, evaluator notes with time spent, and the chat history with the coding agent.

2. The existing system

The server is NestJS with Drizzle ORM on SQLite. All state changes go through `ActionsService.execute`, which: (1) checks idempotency, (2) opens one immediate transaction, (3) checks versions, (4) runs the command in a switch, (5) updates versions and writes audit events, (6) runs `AuditService.review()`, about 25 invariant checks that roll the transaction back on failure. Business problems are signalled by `DomainFailure` (rejection) or `DomainDecision` (a question for the user).

The business is defined in docs/candidate-walkthrough/canonical_business_rules.md (version 2026-10-26.1). Rules I relied on most:

Rule	Meaning
WF-001	Posted entries are never edited; corrections are compensating entries
WF-002	When cash allocation is recomputed (after payment, delivery, correction, return, explicit allocation)
WF-003	Release order on return/correction: cash, then credit, then points
WF-005	One goods charge per delivery confirmation
FUL-001	0 <= returned <= delivered <= shipped
FUL-006	Delivery rules; shipment is Delivered when all dispatched units are confirmed
CUS-001	Settlement and credit funding cannot cross customer accounts
ACC-003	Cash is applied to charges in creation order
CRD-001	Credit is funded from free receipt value
REW-001..004	Reward points: earn, lock, redeem, claw back

3. What I found (current-state rule map)

- Derived state (cash allocation, rewards) was already recomputed idempotently, which is good: after a move or correction, "recompute" gives correct totals.
- Role checks and the reconcile calls were repeated in services, so every new command meant remembering both.
- An action had a single customer in its context, but a payment move touches two.

- Settlement release (WF-003) existed only inside Returns, but delivery correction needs the same logic.
- Database checks blocked negative receipts and negative delivery quantities, so compensating rows needed schema changes.
- Credit funding links receipts to credit notes, which constrains moving a receipt.

4. Design process (the architecture stages)

Stage	File	Essence
Rule map	00-current-state-rule-map.md	Where each rule lives in code; gaps
Draft 1	01-draft-1-first-pass.md	First pass; many options listed
Draft 2	02-draft-2-refined.md	Rules table, reconcile step, reversal rows
Draft 3	03-draft-3-after-research.md	Patterns compared (engine, saga, event sourcing, specification); a thin declarative layer chosen
Final	04-final-architecture.md	What was actually built, limits and metrics

5. The chosen design

5.1 Command rules table

shared/src/command-rules.ts holds one row per command with summary, roles, `reconciles` and rule IDs. It is declared with `satisfies Record<CommandType, CommandRule>`, so adding a command without a row is a compile error. The shell calls `authorize(c, ...rule.roles)` before the switch (Admin always allowed).

5.2 Shell-owned reconcile over touched customers

The action context has a `touched` set that useOrder/useCustomer fill. After the command, if the rule says reconciles, the shell runs allocateCash and the rewards refresh for each touched customer. For each touched customer it also bumps the version, writes an audit event and runs the review. Only the primary customer is version-checked.

5.3 Shared settlement-release primitive

`AccountsService.releaseSettlement(c, charge, newFace, sourceId, released)` lowers a charge and releases excess payment in WF-003 order. Returns was changed to call it, and delivery correction uses it too.

5.4 Derived shipment status

`shipmentStatusFor(awaiting)` returns DISPATCHED while any dispatched unit awaits confirmation, otherwise DELIVERED. Both deliver and correct call a sync step, so a correction and a later confirmation always land in the right state.

5.5 Why the audit stays separate

I considered deriving audit checks from the same rule objects. I kept them independent so they act as a second opinion that catches bugs in the services. I added **account-boundary** (CUS-001) and **receipt-reversal:<id>** checks.

6. Database changes (migration 0006)

- **receipt_reversals**: id, receipt_id (unique), replacement_receipt_id, reason, action_id, created_at.
- **payment_receipts.moved_from_receipt_id**: set on the replacement receipt.
- **deliveries.reverses_id** with check: qty is non-zero, and negative exactly when reverses_id is set.
- The delivery table is rebuilt by SQLite table-copy; I hand-fixed the generated INSERT...SELECT because drizzle-kit referenced a column that did not exist in the old table.

7. FDSK-170 in detail: CORRECT_DELIVERY

1. Authorize KAM (or Admin); load order and shipment; the shipment must be Dispatched or Delivered.
2. For each line: the corrected quantity must be lower than delivered (else NOT_AN_OVERSTATEMENT) and not below returned units (else BELOW_RETURNED, FUL-001); lines must be distinct.
3. Append negative delivery rows reversing the most recent positive deliveries first.
4. Append a negative DELIVERY_REVERSAL charge adjusting the original charge; call releaseSettlement for cash, credit, points above the new charge. If excess remains it fails with CHARGE_MISMATCH.
5. Sync shipment status. The shell then reconciles: released cash is free cash, re-applied by cash priority; rewards refresh locks the points for missing units.

Result for the ticket example: 10 delivered and paid becomes 6 delivered; charge is 6 units; 4 units worth of cash is free; 4 units worth of points locked. Later, Warehouse confirms 4: a new charge appears, free cash pays it, status is Delivered, points unlock.

8. FDSK-169 in detail: MOVE_PAYMENT

1. Authorize Accounts (or Admin). Find a live (not reversed) receipt of the source customer, else RECEIPT_NOT_FOUND.
2. Reject: same customer (SAME_CUSTOMER), unknown target (NOT_FOUND), receipt used to fund a credit note (RECEIPT_FUNDS_CREDIT).
3. Add negative allocations that cancel every allocation of the receipt (compensating rows).
4. Insert a replacement receipt on the target with the same amount, reference and created_at, and moved_from_receipt_id set.
5. Insert the receipt_reversals link; add the target to the touched set.
6. The shell reconciles both customers: source charges reopen and may be re-paid from its other free cash; the target's cash settles its charges; rewards follow, including clawback if points from the moved cash were already redeemed.

9. Decisions and trade-offs

Decision	Reason	Cost
Released cash stays free cash	Simple; matches ACC-003; pays for the missing units later	Customer may briefly hold unapplied cash

Decision	Reason	Cost
Cannot move credit-funded receipt	Would need credit reversal; avoid guessing	Extra manual step for Accounts
Replacement keeps original time	Preserves cash priority	Timestamp is not the move time; the link table records the move
Target version not checked	One primary aggregate per action keeps the model simple	Concurrent edit to target not detected by the move; audit still checks it
New rules ACC-004, FUL-007	Business terms had no receipt-edit or delivery-correction action	Needs business sign-off
No engine / saga	Single transaction, one team, ledger arithmetic	Rules not editable by non-engineers

10. Tests

server/test/corrections.test.ts has 14 tests on an HTTP-level test desk. After each action the helper asserts the action was applied and every audit check passes. Full suite: 265 tests, 0 failures (251 existing + 14 new). Lint, typecheck and prettier are clean.

11. Metrics

	Files	Non-blank lines	Decision nodes
Upstream baseline	126	14,875	939
After my work	128	16,010	982

Growth is mostly the features and tests; role and reconcile repetition was removed from the services.

12. Git history

Upstream history preserved (47 commits). Then small commits: rule map, draft 1, draft 2, draft 3, rules table and reconcile move, settlement-release extraction, schema and contracts, feature implementation, docs, evaluator notes. Each includes an explanatory message.

12b. The screens

Correct delivery: a button on each dispatched or delivered shipment (Shipments tab of the order, KAM/Admin) opens a form listing each line with the quantity marked delivered and an input for the quantity that actually arrived (not below returned units). Move payment: a Move button on each live payment in the customer Payments tab (Accounts/Admin) opens a form with the correct-customer selector and a reason. A moved payment is shown struck through as moved; the replacement shows where it came from. Verified in a real browser; screenshots in docs/demo/.

13. Known gaps and next steps

- The two screens were checked by hand in a browser; there are no automated UI tests.
- Bonus second solution not built; alternatives compared on paper.
- Credit-funded receipts cannot be moved; target version not checked.

- ACC-004 and FUL-007 need business confirmation.
- Fill in time spent, export chat history, add collaborators (atul@patcorn.in, srijan_sahay@patcorn.in, prince@patcorn.in), confirm the late deadline with Srijan, submit the form.